Research Assistant

A Local-First Research Workflow For Papers, Code, and Technical Writing

Colleague Adoption Proposal

April 29, 2026

Contents

Chapter 1

Executive Summary

Researchers already have enough raw materials: PDFs, browser tabs, note files, code comments, and assistant chats. What they usually lack is a durable, inspectable bridge between those materials. The daily cost of that missing bridge is not dramatic in any one moment, but it accumulates everywhere: re-reading the same paper because its relevance was never recorded cleanly, forgetting caveats that mattered when code was first written, repeating literature searches, reconstructing reviewer-facing explanations from memory, and spending more time on research glue work than on model design, validation, or writing.

'research-assistant' is meant to remove part of that workflow tax. It is a local-first research development package for people who regularly move between papers, notes, code, and technical writing. It is built for one serious researcher first: someone who wants a private, inspectable, file-based workspace rather than a shared hosted platform.

The current release focuses on a bounded but useful problem. It helps a researcher ingest papers, inspect source and parser evidence, preserve provenance and review state, discover nearby work, export trusted context for writing or coding, and selectively share reviewable artifacts through Git. It is not a shared department server, not a database-backed collaboration product, not a hosted UI, and not a default live-provider workflow.

The package is persuasive because it is modest in scope. A colleague can install it locally, initialize a workspace, ingest a paper, inspect the result, and decide what to trust without depending on a service operator or central infrastructure. The local-first design keeps the research trail visible, backups straightforward, and failure modes easier to understand.

The before/after contrast is simple. Without a tool like this, paper context is fragmented across PDFs, notes, code comments, and ephemeral assistant memory. With it, the most important parts of that context become structured, reviewable, and reusable across later coding and writing work. The package does not replace judgment, and it does not pretend that machine extraction is proof. Instead, it reduces context loss and manual literature bookkeeping for people who already do serious technical reading and implementation work.

1.1 Current release scope

The current release target is:

- one local researcher first;

- local filesystem storage;
- offline/provider-disabled default workflows;
- Git-based sharing by repository exchange and explicit merge/import;
- explicit review status and provenance rather than silent trust.

The package deliberately does *not* promise:

- shared backend infrastructure;
- shared database writes;
- hosted UI, SSO/RBAC, or real-time collaboration;
- parser scientific accuracy certification;
- automatic promotion of generated artifacts into accepted conclusions.

1.2 Why a colleague should care

A technically skeptical colleague usually does not need another generic paper manager or another generic AI interface. The useful gap is narrower and more costly: preserving an auditable bridge between literature reading and local implementation work.

That gap shows up in familiar ways:

- papers are downloaded, skimmed, and then lost in folders;
- context about why a paper mattered disappears between coding sessions;
- implementation decisions drift away from the literature that motivated them;
- later writing requires reconstructing claims, caveats, and nearby work from memory;
- assistant conversations produce useful draft thinking but not durable, inspectable research state.

Those frictions are not just annoying. They create repeated work. A researcher re-reads papers whose relevance was never captured properly, restates old reasoning to assistants, loses weakly documented caveats, and spends time cleaning up brittle literature notes that never became trustworthy local artifacts.

The package helps by turning that invisible workflow tax into something more manageable. Instead of hoping that relevance, provenance, and uncertainty will still be reconstructible later, the user gets a local, structured, reviewable workspace. That means fewer avoidable mistakes, less repeated explanation, cleaner handoff from papers to code and writing, and a better chance that the next session starts from inspected evidence instead of vague memory.

Chapter 2

The Concrete Workflow Problem

A serious research workflow is usually not blocked by a lack of PDFs. It is blocked by weak continuity between reading, implementation, and writing.

A researcher may begin with a seed paper, inspect code, try a derivation, follow some citations, and eventually draft a note or chapter. Each step is reasonable. The problem is that the evidence trail often fragments:

- paper metadata lives in a filename or browser tab;
- extracted text or notes live somewhere else;
- coding decisions live in comments or memory;
- literature relevance is rediscovered from scratch during later writing;
- uncertainty is hidden rather than recorded explicitly.

That fragmentation is expensive because it compounds over time. The researcher spends energy on repeated context reconstruction: searching for the same paper, re-reading the same section, re-explaining the same method to an assistant, recovering why a claim was treated cautiously, or reconstructing which nearby paper actually supported a coding or writing decision.

The failure modes are also familiar and costly. The repository's own validation materials already name them: wrong metadata merge, ambiguous paper title resolution, summary mismatch, weak provenance, and usability gaps severe enough that a workflow may create more cleanup than time savings. Those are not edge cases. They are exactly the kinds of problems that make otherwise promising research tooling hard to trust.

Existing tools usually solve only part of the problem. A PDF reader is good for reading but not for durable paper-to-code traceability. A coding assistant is good at local code context but does not automatically maintain a durable literature record. A note-taking workflow can hold conclusions, but often loses provenance and review status. A generic paper manager can organize files, but it does not necessarily preserve the specific bridge between what a paper claims, what local code does, and what a later technical note needs to say.

The package is designed to close that narrower gap. It is not trying to replace every tool. It is trying to create a reliable local workflow for moving from paper ingestion to inspectable evidence to trusted downstream context, so the researcher spends less time on glue work and more time on actual technical judgment.

Chapter 3

What The Package Does In The Current Release

The current package provides a conservative ingest, review, discover, and export workflow.

3.1 Core capabilities

The implemented release supports:

- local workspace initialization, validation, repair, and configuration;
- paper ingestion from arXiv IDs, local PDFs, and identifying queries;
- structured-source-first inspection when arXiv LaTeX source is available;
- parser readiness checks, parser capability-limit reporting, and parser-benchmark smoke checks;
- explicit review state for local paper records;
- discovery of related, citing, and cited work through scholarly APIs when available;
- duplicate-aware download proposals into a local inbox;
- trusted-context export for downstream writing and coding workflows;
- backup, restore, privacy-status, and release-report workflows;
- Git-sharing hygiene, dry-run merge, explicit apply, and post-merge rebuild.

3.2 What the package promises

The package is strongest where its promises are deliberately narrow:

- structured-source-first for arXiv papers when source is available;
- local-first and file-based behavior;
- conservative handling of uncertainty;
- clear provenance and review status;
- explicit parser capability limits rather than implied reliability;
- remote enrichment that degrades visibly instead of blocking local work;
- no silent final moves for downloaded papers.

3.3 Trust boundary

Generated outputs are review material. Parser outputs, benchmark results, derivation worksheets, experiment records, synthesis proposals, traceability reports, readiness reports, and validation records do not certify mathematical correctness, scientific parser accuracy, or research approval.

That boundary is not a disclaimer attached at the end. It is the core product posture. The value of the package is that it helps a researcher keep evidence, provenance, and uncertainty visible while human judgment remains explicit.

Chapter 4

Why Local-First Is The Right Design

The current release is intentionally local-first and file-based so it remains inspectable and easy to debug.

That design has practical advantages for the intended user:

- the workspace can be inspected with ordinary filesystem tools;
- backups are simple and explicit;
- failure modes are easier to reason about than in opaque hosted systems;
- the package can be useful without depending on central infrastructure;
- a user can choose when and how to share artifacts rather than being forced into a default shared state.

For this release, Git is the sharing layer rather than a transactional database. A researcher can publish or exchange a repository that contains only shareable artifacts, another researcher can inspect it, run repository hygiene checks, dry-run a workspace merge, and apply a safe subset only with explicit confirmation.

This makes collaboration possible without pretending that Git solves semantic research disagreement. Merge conflicts and accepted-audit conflicts remain human review events.

Chapter 5

How A Real Assistant Conversation Works

The most persuasive way to understand the package is not to read a command list. It is to imagine what a colleague would actually type after the MCP is installed, and what the assistant should do in response.

The package is valuable because those conversations are not just generic chat. They are grounded in a local, inspectable research workspace. When a user asks for a paper audit, a trusted export, or a safe Git-sharing check, the assistant should trigger documented package capabilities that preserve provenance, surface uncertainty, and keep human review visible.

This chapter therefore shows prompt-to-capability behavior rather than a magical black box. The examples below are grounded in the documented workflows in the repository. They show the kinds of requests a colleague could realistically make, what tool or command sequence those requests should trigger, and what sort of result the user should expect.

5.1 Prompt patterns and the documented behavior they should trigger

5.1.1 Local setup, demo, privacy, and diagnostics

Prompt: "Initialize a workspace for me here."

Likely behavior:

```
ra init
```

Why this fits: the workspace must exist before any research action.

Review boundary: low; the user should still inspect the workspace root.

Prompt: "Show me whether this install is healthy."

Likely behavior:

```
ra doctor
ra doctor --matrix
```

Why this fits: these commands surface local readiness, optional tools, and workflow limits.

Review boundary: low; diagnostics are informative, not approval.

Prompt: "Run the safest first-use demo."

Likely behavior:

```
ra --root /tmp/research-assistant-demo demo setup
ra --root /tmp/research-assistant-demo demo run
```

Why this fits: the demo gives a deterministic, non-sensitive first experience before a real paper is involved.

Review boundary: no research judgment required.

Prompt: "Tell me whether the privacy defaults are still safe."

Likely behavior:

```
ra privacy status
```

Why this fits: it confirms the offline/provider-disabled posture before real use.

Review boundary: low; the user still decides whether the posture is sufficient.

5.1.2 Parser readiness and capability checks

Prompt: "Check parser readiness before I trust a PDF workflow."

Likely behavior:

```
ra doctor --matrix
ra parser-tool-matrix
```

Why this fits: these commands surface local tool availability and workflow capability limits before ingest.

Review boundary: yes; the user decides whether fallback quality is acceptable.

Prompt: "Benchmark parser behavior quickly before I rely on extraction."

Likely behavior:

```
ra parser-benchmark-smoke
```

Why this fits: it provides a bounded fixture-level smoke check rather than pretending certification.

Review boundary: yes; the result is still a capability signal, not proof of parser accuracy.

5.1.3 Source-first paper audit and PDF fallback

Prompt: "Use source-first extraction for this arXiv paper."

Likely behavior:

```
ra ingest --arxiv-id ... --query "..."
ra source-fetch --arxiv-id ...
ra source-show --paper-id ...
ra source-sections --paper-id ...
ra source-equations --paper-id ...
```

Why this fits: for mathematical audit, source is the higher-trust substrate when available.

Review boundary: yes; source artifacts are still evidence, not proof.

Prompt: "I only have a local PDF. Show me what the parser can recover."

Likely behavior:

```
ra ingest --pdf /path/to/paper.pdf --query "..."
ra parse-pdf --pdf /path/to/paper.pdf
ra parser-tool-matrix
```

Why this fits: it falls back honestly to parser output and capability limits.

Review boundary: yes; parser output is explicitly not certified.

5.1.4 Review, trusted export, and literature exploration

Prompt: "Show me this paper and tell me whether it is ready to trust."

Likely behavior:

```
ra show --paper-id ...
ra review-show --paper-id ...
```

Why this fits: it combines the local record with explicit review state and provenance.

Review boundary: yes; trust is always a human decision.

Prompt: "Export only approved context for my drafting session."

Likely behavior:

```
ra review-mark --paper-id ... --status approved
ra export-context --review-status approved --output /tmp/paper_context.json
```

Why this fits: it produces bounded downstream context instead of exporting everything blindly.

Review boundary: yes; approved status is the gate.

Prompt: "Find nearby work on transport maps and HMC."

Likely behavior:

```
ra discover --query "transport maps hmc"
ra citation-neighborhood --paper-id ...
```

Why this fits: it uses both topic discovery and citation structure to expand the local view.

Review boundary: yes; nearby work can still be irrelevant or degraded.

Prompt: "Download open-access candidates, but do not silently add them to the library."

Likely behavior:

```
ra download-paper --query "transport maps hmc"
ra inbox-list
ra inbox-show --proposed-name ...
```

Why this fits: it preserves the no-silent-final-moves posture through an inbox review step.

Review boundary: yes; inbox proposals are explicitly reviewable.

5.1.5 Git sharing, backup, and safety

Prompt: "Is this workspace safe to share with a colleague?"

Likely behavior:

```
ra repository-hygiene check --strict
```

Why this fits: it checks for private/generated files and policy violations before sharing.

Review boundary: yes; sharing is a trust-boundary event.

Prompt: "Preview what would happen if I merged this other research repo."

Likely behavior:

```
ra workspace merge --source /path/to/other/repo --dry-run
```

Why this fits: it keeps merge conservative by showing the proposed actions first.

Review boundary: yes; dry-run inspection is required before apply.

Prompt: "Create a backup, but do not restore anything yet."

Likely behavior:

```
ra backup create
ra backup inspect --path /path/to/backup.tar.gz
ra --root /tmp/restore-check backup restore --path /path/to/backup.tar.gz
```

Why this fits: it preserves the explicit-safety posture around backup and restore.

Review boundary: yes for restore; backup creation itself is low-risk.

5.2 Why these prompts do not all map to one command

Some requests are naturally one-tool requests: workspace initialization, privacy-status checks, repository-hygiene inspection, or a single trusted export. Those are easy because the user goal is narrow and the trust boundary is clear.

Other requests are naturally multi-step. "Audit this known paper for my chapter" should not be a one-shot black box. A realistic assistant should ingest the paper, prefer source if available, show sections or equations, surface review status, and only then help the user decide what to trust or export. Similarly, "Find nearby work" often means discovery plus citation-neighborhood inspection, not a single command with implied certainty.

This is one of the package's main strengths. A generic assistant chat can feel fluid, but it often hides where context came from or how uncertain it is. Here, assistant convenience is paired with explicit local artifacts, visible limits, and clear review boundaries. Generated artifacts remain review material, not approval, and the assistant should surface that boundary rather than quietly acting as if a derivation, synthesis, or traceability record were final truth.

5.3 When the assistant should *not* trigger a tool

One of the most important colleague-facing distinctions is that not every good question should trigger MCP behavior. Sometimes the right answer is ordinary reasoning or explanation from

the assistant, using the conversation context that already exists, rather than a trip through the local research workspace.

A tool should usually trigger when the user is asking for:

- current local research state;
- source-grounded paper evidence;
- explicit review status or trusted export;
- parser readiness or capability limits;
- repository hygiene, backup, restore, or merge-preview safety checks.

A tool should usually *not* trigger when the user is asking for:

- a conceptual explanation that does not depend on local artifacts;
- generic advice on how to approach a research problem;
- prose improvement to text already present in the conversation;
- high-level comparison or brainstorming that does not require local workspace inspection.

5.3.1 Examples that may seem like tool triggers, but usually are not

Prompt: "What is the difference between HMC and RMHMC?"

Why this usually should not trigger a tool: this is a conceptual question. Unless the user explicitly asks to compare local paper records or audit a chapter against a stored paper, the assistant can answer directly.

Prompt: "Can you improve this paragraph?"

Why this usually should not trigger a tool: if the paragraph is already in the conversation, this is primarily a writing task. A tool should only be used if the user explicitly asks to ground the revision in approved local paper context.

Prompt: "Should I trust normalizing flows for this project?"

Why this usually should not trigger a tool automatically: this is too broad and judgment-heavy to silently convert into discovery or synthesis. The assistant should usually clarify whether the user wants local evidence, known paper audit, or high-level conceptual advice.

Prompt: "Summarize the NeuTra paper for me."

Why this may or may not trigger a tool: if the user wants a general summary, the assistant may answer directly. If the user asks for a summary that matches the local reviewed record or exact source-grounded equations/sections, then ingest/source tools should trigger.

Prompt: "What would be a good literature review structure?"

Why this usually should not trigger a tool first: this is often an ideation question. It should only become a tool-backed workflow if the user asks to build that structure from local approved records or a known citation neighborhood.

This distinction matters because the package is most valuable when it adds local, inspectable evidence. It is less useful if it is triggered for every general question that an assistant could answer well without touching the workspace.

5.4 Realistic conversation 1: source-first audit of a known paper

User goal: verify what a known paper explicitly claims before revising technical writing or code.

Conversation transcript:

User: "I need to audit the NeuTra paper for a chapter revision. Use source-first extraction if possible, and show me the equations before I trust the summary."

Assistant: "This is a known arXiv paper, so the higher-trust path is to fetch the source first, inspect the source-derived structure, and only then use the local record for downstream writing support."

Likely tool or command sequence:

```
ra --root /tmp/ra-neutra-audit init
ra --root /tmp/ra-neutra-audit ingest --arxiv-id 1903.03704 \
  --query "NeuTra-lizing Bad Geometry in Hamiltonian Monte Carlo Using Neural
      Transport"
ra --root /tmp/ra-neutra-audit source-fetch --arxiv-id 1903.03704
ra --root /tmp/ra-neutra-audit source-sections --paper-id paper_arxiv_1903_aa33312a
ra --root /tmp/ra-neutra-audit source-equations --paper-id paper_arxiv_1903_aa33312a
ra --root /tmp/ra-neutra-audit source-show --paper-id paper_arxiv_1903_aa33312a
```

Assistant: "The source-derived structure is available. You can now inspect sections, equations, and citations from the LaTeX source rather than relying only on PDF heuristics. This is stronger for chapter audit, but it is still evidence that requires human interpretation."

Why this is persuasive:

- it shows a concrete mathematical workflow;
- it demonstrates why source-first is not a slogan but a practical trust move;
- it keeps the review boundary explicit.

5.5 Realistic conversation 2: ingest, inspect, review, and export trusted context

User goal: bring a paper into the workspace, inspect what was extracted, and export only trusted local context for a later coding or writing session.

Conversation transcript:

User: "Ingest this paper and tell me whether it looks safe enough to use in a drafting session."

Assistant: "The conservative path is to ingest it, inspect the record and review state, and only export approved context. That prevents unreviewed material from becoming downstream trusted input by accident."

Likely tool or command sequence:

```
ra --root ~/ra-workspace ingest \
  --pdf ~/papers/example.pdf \
  --query "paper title"
ra --root ~/ra-workspace show --paper-id paper_example
```

```
ra --root ~/ra-workspace review-show --paper-id paper_example
ra --root ~/ra-workspace review-mark --paper-id paper_example --status approved
ra --root ~/ra-workspace export-context \
  --review-status approved \
  --output /tmp/paper_context.json
```

Assistant: "The important distinction is that the export should come from approved records, not from everything that happened to be ingested. That is how the package reduces cleanup while preserving trust discipline."

Why this is persuasive:

- it shows a realistic day-to-day workflow, not just a demo;
- it ties export behavior directly to review status;
- it makes the value of trusted-context export concrete.

5.6 Realistic conversation 3: discovery, nearby work, and safe sharing

User goal: explore nearby literature, then check whether the resulting workspace is safe to share or merge with a colleague.

Conversation transcript:

User: "Find nearby work on transport maps and HMC, but do not pretend the discovery is reliable if the sources are degraded. Then show me the safest way to share the resulting workspace."

Assistant: "That request naturally splits into two bounded workflows: first discovery plus citation-neighborhood inspection, then a repository-hygiene and dry-run merge check for sharing. The important behavior is visible degradation and explicit safety review, not silent automation."

Likely tool or command sequence:

```
ra --root ~/ra-workspace discover --query "transport maps hmc"
ra --root ~/ra-workspace citation-neighborhood --paper-id neutra_hmc
ra --root ~/ra-workspace inbox-list
ra repository-hygiene check --strict
ra workspace merge --source /path/to/other/repo --dry-run
```

Assistant: "Discovery may come back available, empty, or unavailable. That is useful because it distinguishes lack of evidence from tool failure. Before sharing, repository hygiene and dry-run merge keep private/generated files and accepted-audit conflicts visible."

Why this is persuasive:

- it shows how the package handles uncertainty honestly;
- it combines literature expansion with a concrete collaboration safety story;
- it demonstrates that assistant-driven use still respects explicit review gates.

Chapter 6

Concrete Example Workflows

6.1 Example 1: Ingest a paper and inspect what is safe to trust

A common first-use case is simple: a researcher has a paper that may matter for a project, but does not want to trust a vague memory or a one-shot summary.

The workflow can begin with a local workspace and a single ingest:

```
ra --root ~/ra-workspace init
ra --root ~/ra-workspace ingest \
  --pdf ~/papers/example.pdf \
  --query "paper title"
ra --root ~/ra-workspace show --paper-id paper_example
ra --root ~/ra-workspace review-show --paper-id paper_example
```

The important point is not that the first output is magically final. The useful behavior is that the researcher can inspect:

- metadata provenance;
- parser/source extraction status;
- confidence and review state;
- what still requires human review.

This is a better starting point than a disconnected PDF folder because the uncertainty is visible rather than hidden.

6.2 Example 2: Expand nearby literature without losing the evidence trail

Suppose the researcher has found one promising seed paper and wants a fast but reviewable picture of nearby work.

```
ra --root ~/ra-workspace find --query "Neural Transport HMC"
ra --root ~/ra-workspace citation-neighborhood \
  --paper-id neutra_hmc
ra --root ~/ra-workspace discover --query "transport maps hmc"
ra --root ~/ra-workspace inbox-list
```

This does not turn literature discovery into an automatic truth engine. It helps a researcher see what is nearby, what is available, and what needs explicit inspection. Remote enrichment may be empty or unavailable; the package surfaces that status rather than pretending success.

That behavior is useful for real work because it reduces manual graph traversal while keeping the review boundary intact.

6.3 Example 3: Export trusted context for writing or coding

A later pain point is not finding papers, but recovering trusted local context for a coding session or technical note.

```
ra --root ~/ra-workspace review-mark \
  --paper-id neutra_hmc \
  --status approved
ra --root ~/ra-workspace export-context \
  --review-status approved \
  --output /tmp/paper_context.json
```

The value here is practical. Once the researcher has approved a subset of local records, the package can produce a conservative context export for downstream use in writing or coding tools.

That export is intentionally bounded. It should come from reviewed records, not from everything that happened to be ingested. The package therefore helps with speed *and* discipline.

6.4 Nontrivial showcase: DSGE survey chapter from a NeuTra seed

The earlier assistant-conversation chapter shows what a realistic MCP-guided audit looks like in dialogue form. This later showcase keeps only the broader workflow summary: a researcher wants to compare normalizing-flow and neural-transport architectures for difficult posterior geometry and needs a durable local workspace for seed papers, citation neighborhoods, architecture taxonomies, chapter links, code experiments, review notes, and trusted context exports.

The package does not replace the researcher or write the survey automatically. Its job is to keep those artifacts reviewable, traceable, and reusable. For mathematically sensitive material, the higher-trust path is still source-first inspection of known papers, with discovery and synthesis treated as secondary, reviewable aids.

```
ra --root ~/ra-workspace ingest \
  --query "NeuTra-lizing Bad Geometry in Hamiltonian Monte Carlo Using Neural
      Transport"
ra --root ~/ra-workspace ingest \
  --query "Normalizing Flows for Probabilistic Modeling and Inference"
ra --root ~/ra-workspace citation-neighborhood \
  --paper-id neutra_hmc --limit 25
ra --root ~/ra-workspace synthesis propose \
  --paper-id neutra_hmc --kind survey_chapter_outline
```

```
ra --root ~/ra-workspace traceability build --paper-id neutra_hmc
ra --root ~/ra-workspace export-context \
  --review-status approved \
  --output /tmp/dsge_flow_chapter_context.json
```

Chapter 7

Scope Boundaries And Non-Goals

A colleague should adopt the package for what it already does well, not for what it might become later.

7.1 Current boundaries

This release is:

- private local use rather than shared departmental deployment;
- Git sharing by repository or snapshot rather than real-time collaboration;
- offline/provider-disabled by default;
- strongest on local auditability, provenance, and trusted-context export.

7.2 Non-goals for the current release

This release does not aim to provide:

- a GUI-first rewrite;
- a database requirement;
- bulk scraping or broad web crawling;
- silent auto-organization of a paper library;
- high-stakes claim verification without explicit evidence support.

7.3 Known limits that matter to adoption

Colleagues should know the practical limits up front:

- parser quality depends on local optional tools and source/PDF quality;
- parser-tool availability can be checked, but parser scientific accuracy is not certified;
- Git-sharing performance evidence is currently synthetic at the 'synthetic_git_1000' tier unless a non-sensitive real corpus is explicitly recorded;
- real colleague onboarding, real macOS validation, and real minimal parser-tool validation remain manual release gates.

Chapter 8

Technical Credibility Without The Monograph

The package is persuasive only if technically serious readers believe the design is disciplined.

8.1 Structured local artifacts and provenance

The workspace keeps local records for metadata, summaries, reviews, source artifacts, downloaded-paper proposals, and exportable trusted context. This is not just a storage choice. It means the user can inspect where information came from and what status it currently has.

8.2 Source-first audit posture

For arXiv papers, LaTeX source is preferred when available. That is a strong and practical design choice for mathematical material because source often provides a better audit substrate than PDF parsing alone. The package therefore treats PDF parser output as fallback and cross-check material rather than as a silently trusted primary source.

In practice, a good rule is: if the paper is known and source is available, use 'source-fetch' first when the task is chapter audit, derivation checking, code-literature alignment, or equation-level review. That workflow is especially useful when the researcher needs to verify what a paper explicitly claims before rewriting technical documentation or implementation notes.

8.3 Explicit parser limits

The package is careful about parser claims. Commands such as 'ra doctor –matrix', 'ra parser-tool-matrix', and 'ra parser-benchmark-smoke' exist to surface tool availability, workflow readiness, fixture smoke behavior, and capability limits. That is more trustworthy than pretending equations, citations, or section structure are always reliable.

8.4 Complements existing coding assistants

The package becomes more useful when paired with existing assistant workflows. Its role is to preserve durable local research state — reviewed paper records, source-derived evidence, discovery results, and trusted exports — that an assistant or script can use later.

Chapter 9

A Practical Adoption Path

A colleague does not need to commit to a department-wide workflow to evaluate the package. The sensible adoption path is a small local pilot, and that is part of its appeal. The package is not asking the user to bet on a future platform before seeing value. It is asking the user to try a bounded workflow on one machine and judge whether it saves time or creates more cleanup.

9.1 Start from a wheel or local checkout

```
python -m pip install research_assistant-0.1.0-py3-none-any.whl
ra version
ra --root ~/ra-workspace init
ra --root ~/ra-workspace doctor
```

A developer can also install from a checkout with 'python -m pip install .', but the colleague-facing path is the wheel and first-workspace flow.

The attraction here is practical: a user can test whether the package fits their own research habits before changing anything bigger. If it helps them preserve paper context, review evidence more carefully, and write or code with less reconstruction overhead, then the value is already proven locally.

9.2 Run the demo before trusting a real workflow

```
ra --root /tmp/research-assistant-demo demo setup
ra --root /tmp/research-assistant-demo demo run
ra --root /tmp/research-assistant-demo release-report
```

This gives the user a deterministic, non-sensitive first experience before any real paper is involved. It is a low-risk way to decide whether the tool feels like genuine workflow leverage rather than another source of cleanup.

9.3 A release-proven use case: source-first NeuTra audit

The release was exercised on a realistic research task: auditing a DSGE/HMC monograph chapter built around *NeuTra-lizing Bad Geometry in Hamiltonian Monte Carlo Using Neural Transport*. The detailed assistant-style conversation for this use case now appears earlier in the proposal. This section keeps the concise adoption takeaway: the package is already genuinely useful for a known-paper audit with source-grounded inspection and downstream writing support.

Representative commands:

```
ra --root /tmp/ra-neutra-audit init
ra --root /tmp/ra-neutra-audit ingest --arxiv-id 1903.03704 \
  --query "NeuTra-lizing Bad Geometry in Hamiltonian Monte Carlo Using Neural
      Transport"
ra --root /tmp/ra-neutra-audit source-fetch --arxiv-id 1903.03704
ra --root /tmp/ra-neutra-audit find --query "NeuTra"
ra --root /tmp/ra-neutra-audit source-sections --paper-id paper_arxiv_1903_aa33312a
ra --root /tmp/ra-neutra-audit source-equations --paper-id paper_arxiv_1903_aa33312a
ra --root /tmp/ra-neutra-audit source-show --paper-id paper_arxiv_1903_aa33312a
```

What this illustrates in practice:

- source-first inspection is already useful for mathematical literature audits;
- structured source outputs make chapter and code verification easier than PDF-only review;
- reviewed context can then be exported into later writing or coding workflows.

What it does *not* illustrate:

- automatic survey writing;
- automatic mathematical certification;
- reliable open-ended literature discovery without human review.

This is exactly the level at which the current release should be presented to colleagues: strong for local, source-grounded audits of known papers, and honest about where human judgment still dominates.

9.4 Use support commands safely

If something fails, the support boundary is explicit: share diagnostics, not research data.

Useful commands include:

```
ra version
ra doctor --matrix
ra platform-status
ra privacy status
ra parser-tool-matrix
```

This is another adoption advantage of the local-first posture: diagnosis can be performed on a demo or empty workspace without exposing actual research content.

Chapter 10

Skeptical Questions And Straight Answers

10.1 Why not just keep using a PDF reader and a note folder?

Because the real cost is not reading alone. The cost is reconstructing later why a paper mattered, what was trusted, what was only provisional, and how the paper related to code or writing. A local structured workflow is useful when that continuity matters, especially when the alternative is repeated re-reading, repeated explanation, and weakly documented technical judgment.

10.2 Why not just use Claude Code directly?

Claude Code is very useful for code and local context, but it does not by itself become the durable research record. The package gives the user a stable local workspace of summaries, provenance, review state, source artifacts, and trusted exports that can then be used in Claude Code more effectively.

A good way to say this is: assistants are already useful, but without a durable research memory layer their usefulness plateaus. They help with the current window of context; this package helps preserve the context that should survive past the current window.

10.3 Why not just use a generic paper manager?

A generic paper manager can help organize files, but it usually does not solve the specific research-engineering problem here: trustworthy paper-to-code and paper-to-writing continuity. The issue is not only storing papers. It is keeping provenance, review status, source-grounded evidence, and downstream trusted context in a form that remains usable during implementation and writing.

10.4 Is this trying to automate research judgment?

No. The package is deliberately conservative. Generated and parser-derived artifacts remain review material, and human approval remains explicit.

10.5 Is this already a shared research platform?

No. The current release is intentionally smaller and more honest than that. It is a local-first individual workflow with Git-based sharing, not a hosted multi-user system.

10.6 Why is the bounded scope a strength rather than a weakness?

Because the package is most credible where it already changes daily work without pretending to solve everything. It is small enough to install locally, inspect, and validate against real research tasks, but complete enough to matter for literature triage, source-first audit, review discipline, and trusted-context export. That makes it easier to evaluate honestly: does it save time, reduce cleanup, and preserve context better than the status quo?

Chapter 11

Future Extensions

Future work may extend the package, but those possibilities should not be confused with the current adoption case.

Possible later directions include:

- stronger parser and source benchmarks on gold corpora;
- richer sanitized real-corpus scalability evidence;
- optional governed provider workflows;
- more developed team conventions around artifact exchange.

Those are not prerequisites for the current package to be useful. The present value proposition is that the local tool is already inspectable, bounded, and practically helpful.

Appendix A

Operational Notes Kept Briefly

A.1 Git sharing remains conservative

Before sharing a repository, a researcher should run:

```
ra repository-hygiene check --strict
ra workspace merge \
  --source /path/to/other/repo \
  --target ~/ra-workspace
ra workspace merge \
  --source /path/to/other/repo \
  --target ~/ra-workspace \
  --apply --confirm-merge
ra workspace rebuild-derived
```

Dry-run is the default. Apply requires explicit confirmation. Accepted-audit conflicts remain human review events.

A.2 Release maturity remains pilot-scoped

Broad release still requires real fresh-reader onboarding, real macOS validation, real minimal parser-tool machine validation, release-owner tag approval, and release-owner publication approval. Until those are recorded, the release remains pilot-scoped.

Closing Note

The honest reason to adopt ‘research-assistant’ is not that it promises a future research platform. It is that, today, it offers a private, local, inspectable, and conservative workflow for carrying paper context into coding and writing work without losing provenance or review discipline.